



School of Future Tech

Case Study Report

STUDENT DETAILS

Student Name: Nishi Chopda

Roll No.: 150096725128

Program: B.Tech Computer Science & Engineering

Semester : II

Project Title: MidiCanvas – MIDI File Parser & Visual Piano Roll

Problem Statement No: 115

Technology: ReactJS

Deployment Platform: [Vercel](#)

Source Code Repository: [GitHub](#)

PROBLEM STATEMENT

The objective of this project is to develop a frontend-only ReactJS application capable of parsing MIDI (.mid) files and visualizing musical note data through an interactive Piano Roll interface. Unlike traditional audio files, MIDI files contain binary event information such as note events, pitch values, velocity, tempo data, and timestamps rather than actual sound recordings.

The application is designed to decode binary MIDI data entirely within the browser and transform it into a structured timeline of musical events. The extracted information is displayed through an interactive Piano Roll interface, enabling users to visualize, analyze, edit, and play MIDI compositions. All processing is performed on the client side without relying on backend servers or external databases.

PROJECT OBJECTIVE

The primary objective of MidiCanvas is to provide an efficient and interactive platform for processing MIDI files directly within the browser. The application enables users to upload MIDI files, decode musical event data, visualize notes on a timeline-based Piano Roll, and generate audio playback using browser-native technologies.

The project also focuses on implementing modern frontend development practices such as state management, local storage, background processing, and responsive user interface design. By combining these technologies, MidiCanvas delivers a Digital Audio Workstation (DAW)-inspired experience while remaining lightweight, scalable, and fully client-side.

TECHNOLOGY STACK

Technology	Purpose
ReactJS	Frontend Framework
JavaScript	Application Logic
Zustand	State Management
Dexie.js	IndexedDB Wrapper
IndexedDB	Local Project Storage
Web Workers	Background MIDI Parsing
Web Audio API	Audio Playback
File API (arrayBuffer)	Binary MIDI File Reading
Vite	Development and Build Tool

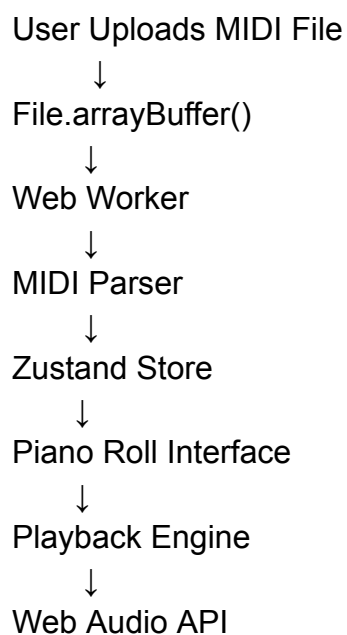
React DOM Elements	Piano Roll Rendering
-----------------------	----------------------

COMPONENT ARCHITECTURE

The architecture of MidiCanvas follows a client-side processing workflow. Users begin by uploading a MIDI file through the browser interface. The uploaded file is converted into binary data using the File API and transferred to a Web Worker for processing.

The Web Worker parses the MIDI structure and extracts note events, timing information, pitch values, and velocity data. The parsed information is then stored within the Zustand state management system. React components consume this state and render the Piano Roll interface, timeline grid, and playback controls. Finally, the playback engine schedules note events and generates audio using the Web Audio API.

System Flow



PROJECT STRUCTURE

src/

— components/	# UI components (editor, library, landing)
— hooks/	# Custom React hooks
— pages/	# Page layouts
— stores/	# Zustand state management
— styles/	# Global styling
— types/	# Type definitions
— utils/	# Helper functions and utilities
— workers/	# Web Workers for MIDI parsing

FEATURES IMPLEMENTED

MIDI File Upload

Users can upload local MIDI (.mid) files directly from their device using the browser's native File API.

Binary MIDI Parsing

Uploaded files are processed using Web Workers to decode note events, timestamps, pitch values, and velocity information without blocking the user interface.

Interactive Piano Roll Visualization

Musical notes are rendered on a timeline-based Piano Roll interface featuring a vertical piano keyboard and editable note blocks.

Playback Controls

The application includes fully functional Play, Pause, Resume, Stop, and Loop controls for navigating MIDI compositions.

Note Editing

Users can interact with notes through selection, movement, resizing, and deletion operations with real-time state updates.

Local Storage

Project data can be stored locally using IndexedDB through Dexie.js, allowing users to preserve their work without requiring a backend database.

Performance Monitoring

Real-time telemetry provides information about playback status, timeline position, and application activity.

Quantization Support

Grid-based alignment utilities help maintain accurate note placement and timing consistency.

WORKING OF THE APPLICATION

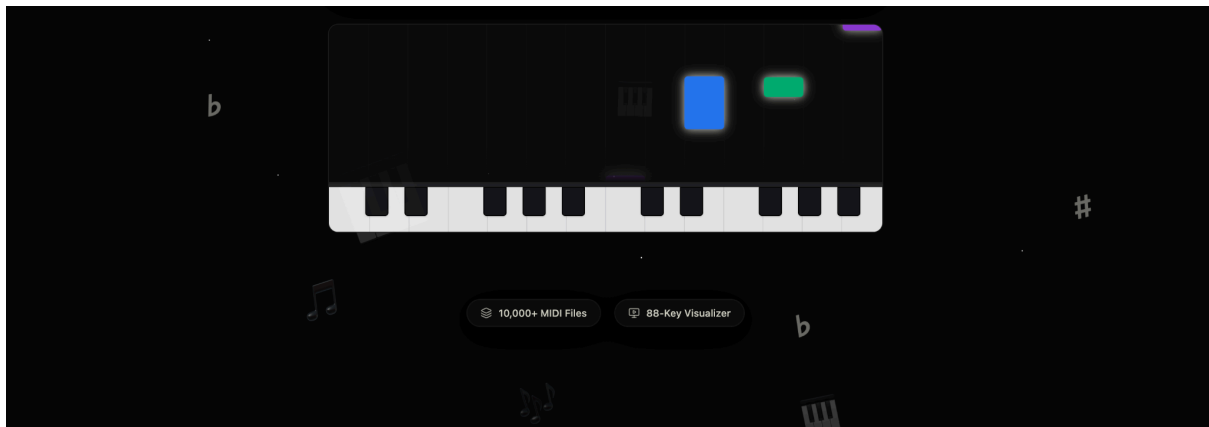
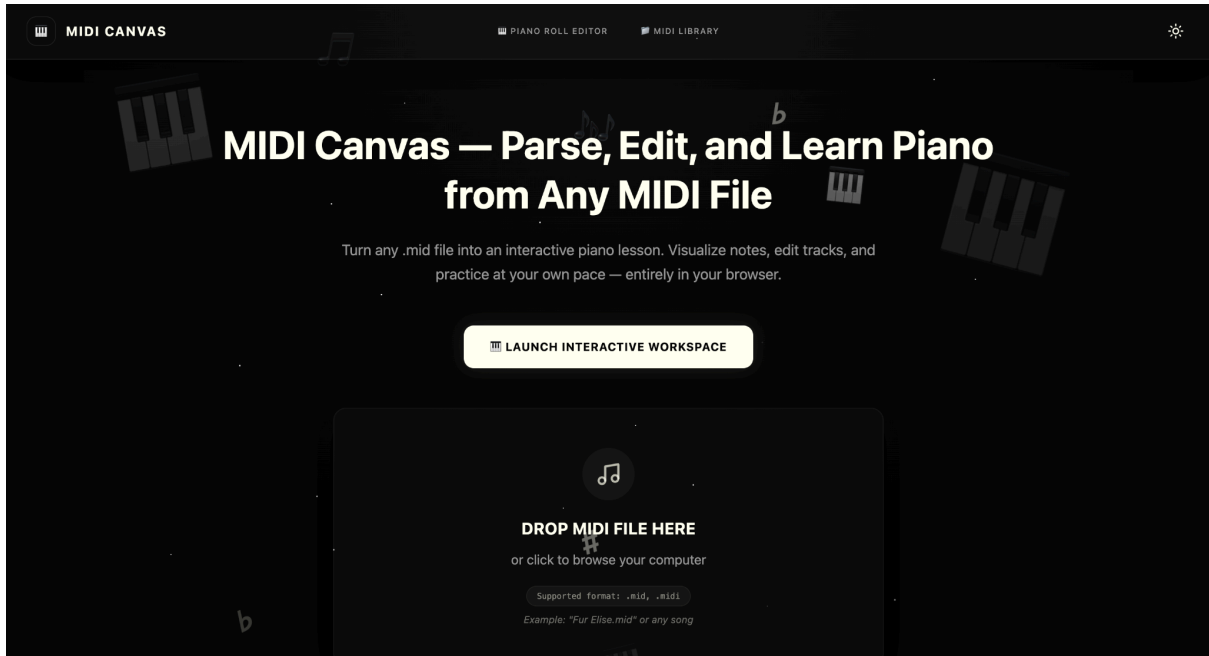
The workflow begins when a user uploads a MIDI file through the application interface. The browser reads the file as binary data using the native File API and converts it into an ArrayBuffer. This binary data is then transferred to a Web Worker where MIDI events are decoded and transformed into structured note objects.

The parsed note information is stored within Zustand and supplied to the Piano Roll interface for visualization. Notes are rendered on a timeline where users can inspect, edit, and manage musical events. During playback, the Web Audio API synthesizes the corresponding sounds while maintaining synchronization with the visual timeline. This architecture enables efficient processing and smooth interaction entirely within the browser.

SCREENSHOTS

Insert screenshots with captions for the following sections:

- Landing Page



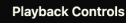
#



Real-Time Audio Playback



b

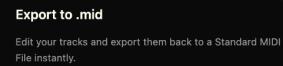
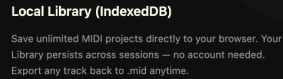
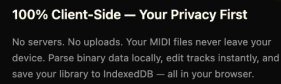


Interactive Piano Roll

Canvas grid + vertical keyboard. Click keys to hear notes.
Drag, stretch, or delete notes.



b

**b**

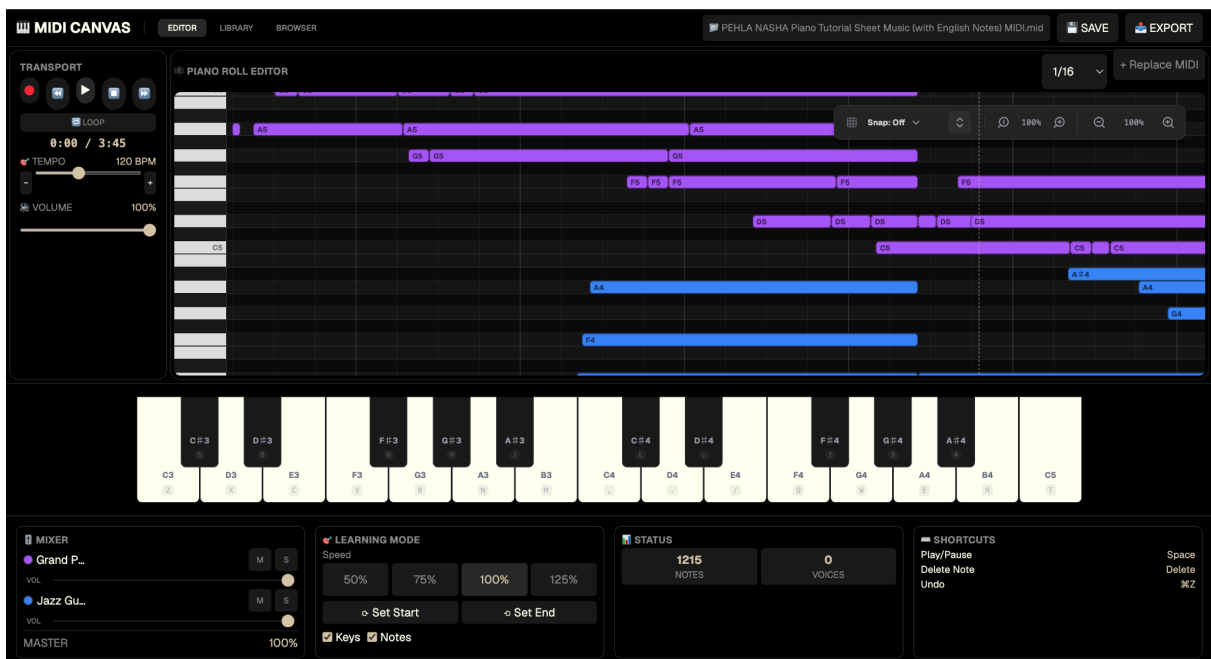
PLAYBACK

Engine decodes binary → note array with pitch, timing, velocity

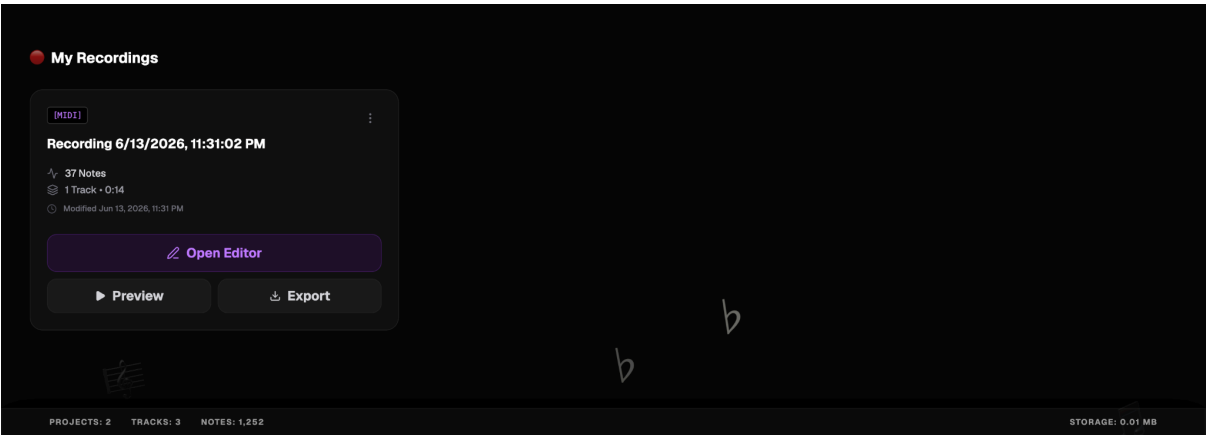
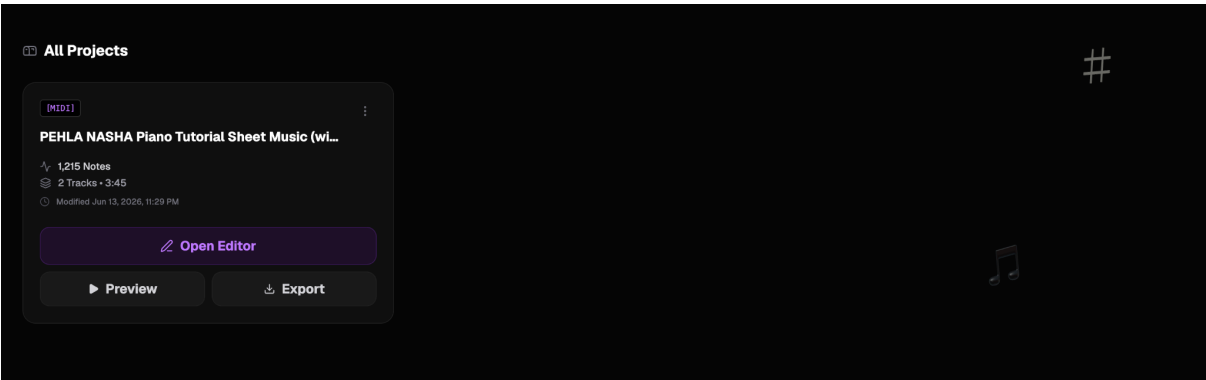
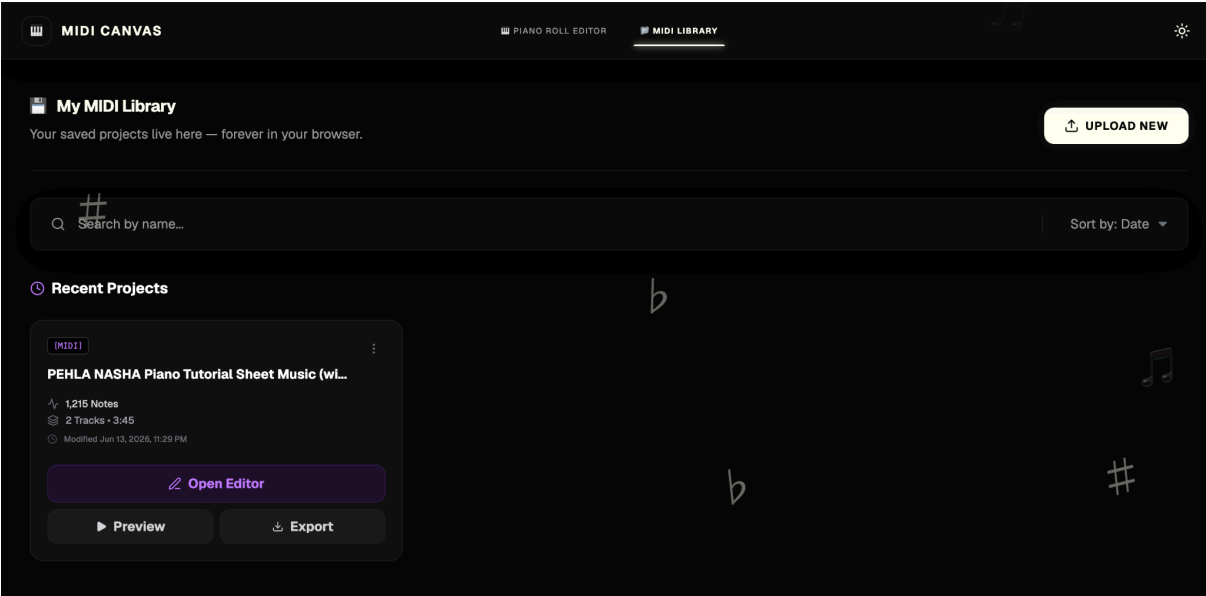
Click, drag, stretch, or delete notes. Snap to grid.

Hear your edits instantly via Web
Audio synthesis





- Library Module



CHALLENGES FACED

MIDI Binary Parsing

Understanding and decoding the binary structure of MIDI files required extensive research into MIDI event formats, timing systems, and track organization.

Timeline Synchronization

Maintaining synchronization between visual note rendering and audio playback was essential to ensure an accurate user experience.

Interactive Note Editing

Implementing note movement, resizing, and deletion functionality required efficient state updates and careful handling of user interactions.

Performance Optimization

Large MIDI files can contain thousands of note events. Efficient rendering strategies and background processing through Web Workers were necessary to maintain smooth application performance.

CONCLUSION

MidiCanvas successfully demonstrates a complete client-side MIDI File Parser and Visual Piano Roll application built using ReactJS. The project fulfills all major objectives of the assigned problem statement, including MIDI file ingestion, binary parsing, note visualization, playback control, local storage, and interactive note editing.

By leveraging modern frontend technologies such as ReactJS, Zustand, Dexie.js, IndexedDB, Web Workers, and the Web Audio API, the application delivers a Digital Audio Workstation-inspired experience entirely within the browser. The project highlights practical implementation of binary data processing, state management, performance optimization, and interactive music visualization techniques.

PROJECT LINKS

Live Demo: <https://midi-canvas.vercel.app>

GitHub Repository: <https://github.com/nishhhi11/MidiCanvas>